In [ ]:

```
import numpy as np
import pandas as pd
import time

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score

# Step 2: Load the Fashion MNIST dataset
print("Loading Fashion MNIST dataset...")

fashion_mnist = fetch_openml(
    name="Fashion-MNIST",
    version=1,
    as_frame=False
)

X = fashion_mnist.data.astype(np.float32)
y = fashion_mnist.target.astype(int)

print("Dataset shape:", X.shape)
print("Number of classes:", len(np.unique(y)))

# Step 3: Normalize pixel values
# Original pixel values range from 0 to 255
X = X / 255.0

# Step 4: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Step 5: Use a subset to reduce computation time
# Select 4000 training images and 1000 testing images
X_train, _, y_train, _ = train_test_split(
    X_train,
    y_train,
    train_size=4000,
    random_state=42,
    stratify=y_train
)

X_test, _, y_test, _ = train_test_split(
    X_test,
    y_test,
    train_size=1000,
    random_state=42,
    stratify=y_test
)

print("\nTraining samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
# Step 6: Experiment with different K values
k_values = [1, 3, 5, 7, 9]

results = []

for k in k_values:
    print("\nTraining KNN with K =", k)

    # Create the KNN classifier
    model = KNeighborsClassifier(
        n_neighbors=k,
        metric="euclidean",
        n_jobs=-1
    )

    # Train the model
    start_train = time.time()
    model.fit(X_train, y_train)
    training_time = time.time() - start_train

    # Predict the classes of test images
    start_predict = time.time()
    y_pred = model.predict(X_test)
    prediction_time = time.time() - start_predict

    # Evaluate the model
    accuracy = accuracy_score(y_test, y_pred)
    f1 = f1_score(
        y_test,
        y_pred,
        average="weighted",
        zero_division=0
    )

    # Store results
    results.append({
        "K": k,
        "Accuracy": accuracy,
        "Weighted F1-Score": f1,
        "Training Time (seconds)": training_time,
        "Prediction Time (seconds)": prediction_time
    })

    print("Accuracy:", round(accuracy, 4))
    print("Weighted F1-score:", round(f1, 4))
    print("Training time:", round(training_time, 3), "seconds")
    print("Prediction time:", round(prediction_time, 3), "seconds")

# Step 7: Display the comparison table
results_df = pd.DataFrame(results)

print("\n========== KNN PERFORMANCE COMPARISON ==========")
print(results_df.round(4).to_string(index=False))

# Step 8: Find the best K based on accuracy
best_index = results_df["Accuracy"].idxmax()
best_k = results_df.loc[best_index, "K"]
best_accuracy = results_df.loc[best_index, "Accuracy"]

print("\nBest K value:", best_k)
```

```
print("Highest accuracy:", round(best_accuracy, 4))

# Step 9: Find the best K based on weighted F1-score
best_f1_index = results_df["Weighted F1-Score"].idxm
```

```
Loading Fashion MNIST dataset...
```

In [ ]: